Rock Paper Scissors Deep Learning

Ximena Pérez Escalante

A01751827

Instituto Tecnológico y de Estudios Superiores de Monterrey

Noviembre 2026

Abstract A través de un modelo de visión computacional de Deep Learning, se busca predecir la clase de una imagen, entre las tres posibles: “rock”, “paper” y “scissors”.

Keywords (palabras clave): Computer Vision, Deep Learning, CNN

Introducción

El juego “piedra, papel o tijeras” es conocido universalmente y ha sido jugado por todas las personas en al menos un momento de sus vidas. A partir de esa popularidad, surge la intención de programar una Red Neuronal Convolucional que pueda predecir, a partir de la imagen de una mano, si está haciendo la forma de una de las tres clases (piedra, papel o tijeras). La primera versión de la interfaz consistirá en subir la imagen y a partir de la predicción del modelo, este tendrá un contraataque para “ganarle” al jugador. Es necesario mencionar que el objetivo no será crear un juego en tiempo real, simplemente es una simulación, donde el usuario dependerá de qué tan bueno sea el modelo en sus predicciones.

Dataset

Se seleccionó el dataset de la plataforma Kaggle, el cual tiene el nombre de “Rock-Paper-Scissors-Dataset”. Este mismo cuenta con un total de 2717 imágenes divididas en tres carpetas (una por cada clase):

- **Rock:** 907 imágenes
- **Paper:** 907 imágenes
- **Scissors:** 903 imágenes

Las imágenes se encuentran en formato PNG y tienen una resolución de 300 x 300px. En ellas se capturaron las manos de tres personas diferentes, desde diferentes ángulos, con el mismo fondo, pero distinta iluminación. Algunos ejemplos de estas variaciones se muestran a continuación:

Rock

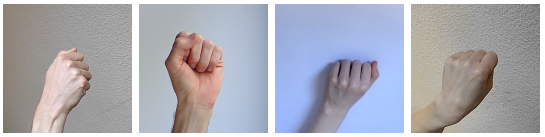


Figura 1: ejemplos de la clase “rock”

Paper

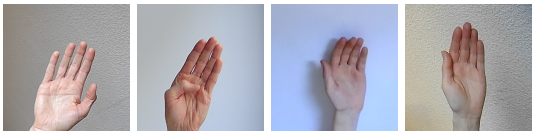


Figura 2: ejemplos de la clase "paper"

Scissors

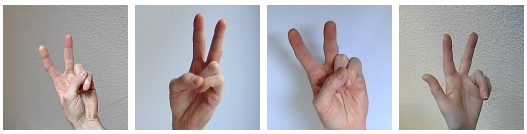


Figura 3: ejemplos de la clase "scissors"

Limitaciones

Las imágenes del dataset seleccionado presentan algunas limitaciones, entre ellas se encuentra la similitud entre ellas, ya que aunque es cierto que la iluminación y los ángulos son diferentes, pero en todas lo único que se ve es la mano sobre un fondo blanco. Por otro lado está el color de piel, ya que todas las personas que fueron fotografiadas son de tez blanca, así que esto puede generar un sesgo en el modelo, donde imágenes de personas de piel morena y oscura, reciban predicciones erróneas.

ETL

Extracción

Se comenzó por descargar el dataset desde Kaggle, en este caso como se está trabajando con imágenes, se programó una función para poder cargarlas a Python y otra para agregarlas al Dataset. Para esto fue necesario asignarle una etiqueta a cada imagen

Transformación

Esta etapa consistió en convertir las imágenes en un tensor RGB (3 canales) de 300x300 pixeles. Posteriormente, estos pixeles pasaron de ser enteros en un rango 0-255 al tipo de dato float en un rango de 0-1. De igual manera, se redujo la dimensión de 300 x 300 pixeles a 150 x 150 pixeles.

Data Augmentation

El objetivo es que el modelo aprenda la forma de la mano sin depender de su orientación, posición o iluminación, lo que reduce el sobreajuste. Las acciones que se realizarán de forma aleatoria son las siguientes:

- **Flip:** volteo horizontal y vertical.
- **Rotación:** gira la imagen en un ángulo aleatorio de hasta ±20% de una vuelta completa, es decir, ±72°.
- **Zoom:** acerca o aleja la imagen hasta un 15%.
- **Contraste:** modifica el contraste por un factor aleatorio entre 0.8 y 1.2.

Carga

División de datos

Para separar las imágenes en "train", "validation" y "test", primero se hace un shuffle de todas, debido a que la cantidad por clase es equitativa.

Se utilizó el siguiente ratio: 70-15-15. 70% train, 15% validation y 15% test. Lo cual equivale a la siguiente cantidad por división:

- Train: 1,903 imágenes
- Validation: 407 imágenes
- Test: 407 imágenes

Redes Neuronales Convolucionales

Estas redes son una arquitectura de Deep Learning diseñada principalmente para datos como imágenes. Funcionan aplicando kernels, también conocidos como filtros. Estos van recorriendo la imagen con la finalidad de identificar diversas características de las mismas, entre las que se encuentran bordes, texturas, patrones y/o formas. Estas características se extraen mediante capas convolucionales y se procesan con funciones de activación. Por otro lado, se tienen capas de pooling, las cuales reducen progresivamente las dimensiones de los datos. Posteriormente, estas características previamente extraídas se utilizan en capas completamente conectadas (Fully Connected), que se encargan de realizar la clasificación. Durante el entrenamiento, la red ajusta automáticamente los pesos de sus filtros mediante algoritmos de optimización y retropropagación (backpropagation), minimizando el error entre las predicciones y los valores reales.

Modelo V1

Para la primera versión del modelo, se decidió comenzar con algo relativamente sencillo, basado en CNNs como AlexNet y VGG al utilizar múltiples bloques de convolución 3x3, alternando pooling y convoluciones, además, de la duplicación de los kernels conforme se reduce la imagen (32 - 64 - 128).

1. Capa de entrada

Define la forma de los datos que recibe la red: un tensor de 150x150x3 (alto, ancho y los tres canales de color RGB) con valores normalizados en el rango [0, 1]. No tiene parámetros, sólo establece las dimensiones con las que trabajan las capas siguientes.

2. Aumento de datos

Este bloque aplica las transformaciones aleatorias a cada imagen durante el entrenamiento. Estas capas no tienen parámetros y Keras las desactiva automáticamente durante la evaluación y la inferencia. La salida conserva la forma 150x150x3.

3. Bloques convolucionales (1 a 4)

La extracción de características se hace con cuatro bloques de la misma estructura. En cada bloque se duplica el número de filtros, hasta 128, mientras que la resolución espacial se reduce a la mitad. Cada bloque contiene tres capas:

- **Conv2D:** aplica filtros de 3x3 que recorren la imagen con paso 1. Cada filtro aprende a detectar un patrón local. El padding de tipo “same” añade bordes para que la salida conserve el mismo alto y ancho que la entrada. La activación ReLU, $f(x) = \max(0, x)$, introduce no linealidad y anula los valores negativos. El número de parámetros es (3 x 3 x canales de entrada + 1) x número de filtros, donde el +1 corresponde al sesgo de cada filtro.

- **Batch Normalization (momentum 0.9):** normaliza las activaciones de cada canal con la media y la varianza del lote, y luego las ajusta con dos parámetros aprendibles (γ y β). Esto estabiliza la distribución de los datos entre capas, acelera el entrenamiento y tiene un ligero efecto regularizador. Además, guarda una media y una varianza móviles (no entrenables) que se usan en la inferencia. Tiene 4 parámetros por canal.
- **MaxPooling2D (2x2):** divide cada mapa de características en regiones de 2x2 y conserva solo el valor máximo de cada una. Así reduce el alto y el ancho a la mitad (redondeando hacia abajo), disminuye el costo computacional y hace al modelo más tolerante a pequeños desplazamientos. No tiene parámetros.

Las características son cada vez más abstractas a medida que se avanza en la red. Los primeros bloques detectan bordes, texturas y cambios de color; los últimos combinan esos patrones en estructuras más complejas, como la forma de los dedos extendidos o del puño cerrado. El cuarto bloque concentra la mayor parte de los parámetros del modelo (57%).

4. Average Pooling 2D

Calcula el promedio de cada uno de los 128 mapas de características de 9x9 y produce un vector de 128 valores, uno por canal. Cada valor indica qué tan presente está una característica en la imagen completa. Reemplaza a la capa Flatten que se usa tradicionalmente. Si se aplanara la salida (9 x 9 x 128 = 10,368 valores), la siguiente capa densa necesitaría 1,327,232 parámetros; con el promedio global necesita solo 16,512. Esto reduce mucho el riesgo de sobreajuste. No tiene parámetros.

5. Dropout (0.5)

Durante el entrenamiento, pone en cero el 50% de los 128 valores, elegidos al azar en cada paso, y multiplica los restantes por $1/(1 - 0.5)$ para mantener la escala. Así la red no puede depender de unas pocas neuronas y tiene que aprender representaciones redundantes y más generales. Se desactiva en la inferencia y no tiene parámetros.

6. Dense (128 unidades, ReLU)

Es una capa totalmente conectada: cada una de sus 128 neuronas recibe los 128 valores de la capa anterior. Combina las características extraídas por los bloques convolucionales para formar una representación orientada a la clasificación. Tiene 128 x 128 + 128 = 16,512 parámetros.

7. Dropout (0.3)

Funciona igual que la capa de Dropout anterior, pero desactiva el 30% de las neuronas. Se usa una tasa menor porque esta capa está más cerca de la salida y tiene menos unidades.

8. Dense (3 unidades, Softmax)

Es la capa de salida, con una neurona por cada clase: “piedra”, “papel” y “tijeras”). La

función softmax convierte los valores de salida z_i en probabilidades:

$$P(clase_i) = e^{(z_i)} / \Sigma\square\ e^{(z\square)}$$

Las tres probabilidades quedan entre 0 y 1 y suman 1. Tiene 128 x 3 + 3 = 387 parámetros.

9. Salida

El resultado es un vector con la probabilidad de cada clase, por ejemplo [0.02, 0.95, 0.03]. La clase predicha es la de mayor probabilidad (argmax), ese valor máximo se interpreta como la confianza del modelo. El servidor de inferencia (serve.py) devuelve además el contraataque que le gana a la clase predicha.

Configuración para el entrenamiento

El modelo se compiló con el optimizador Adam, con una tasa de aprendizaje inicial de 0.001 y la función de pérdida cross-entropy. La métrica de evaluación fue la exactitud (accuracy), sin embargo, no es la más adecuada y se usarán otras como el F1 score.

Resultados

Pérdida

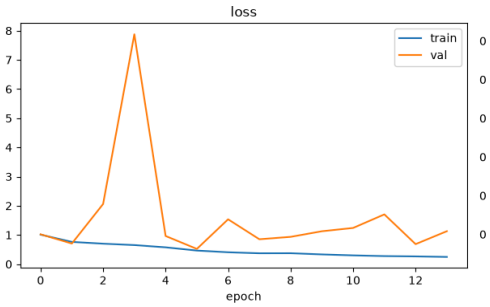


Figura 4: pérdida train vs validation

Accuracy

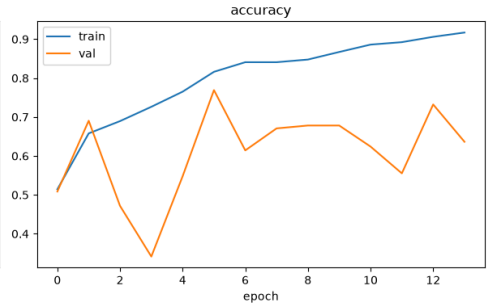


Figura 5: accuracy train vs validation

De acuerdo con las gráficas anteriores, se puede observar que ambas métricas suben y bajan progresivamente, además, la accuracy termina en un valor de aproximadamente el 60% en validación, lo cual representa una diferencia de -40% con respecto al entrenamiento.

Referencias

https://www.kaggle.com/datasets/alexandredj/rock-paper-scissors-dataset

AlexNet

VGG